

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное образовательное учреждение высшего образования
САНКТ – ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель

должность, уч. степень, звание

подпись, дата

Д.О. Шевяков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №8

Абстрактные классы

по курсу: Основы программирования

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4321

подпись, дата

Г.В.Буренков

инициалы, фамилия

Санкт-Петербург 2024

СОДЕРЖАНИЕ

1 Цель работы.....	3
2 Задание.....	3
3 Описание работы программы	4
4 Результаты работы программы	10
5 Вывод	11
Приложение А. Листинг Программы.....	12

1 Цель работы

Изучить работу с абстрактными классами и методами. Реализовать наследование и переопределение в производных классах. Освоить модификатор «sealed» и переопределение Component

2 Задание

Создать абстрактный класс и реализовать наследование: добавить общие свойства и методы в абстрактный класс, определить их реализацию в дочерних.

3 Описание работы программы

Программа реализует работу с абстрактными классами и демонстрирует их использование на примере автомобильных компонентов. Абстрактный класс `Component` представляет общую структуру для всех компонентов автомобиля и содержит общие свойства и методы. В данном случае свойство `Condition` используется для хранения состояния компонента, которое может быть изменено через метод `ChangeCondition`. Метод проверяет корректность переданного состояния, разрешая только значения "Новое" или "Сломано". При изменении состояния на "Сломано" вызывается событие `ComponentBroken`, сигнализирующее о поломке компонента.

```
Ссылка: 1
public abstract class Component
{
    // Абстрактный класс Component, который реализует общие методы и события для всех компонентов автомобиля
    // Свойство Condition хранит текущее состояние компонента, по умолчанию "Новое"
    Ссылка: 8
    public string Condition { get; protected set; } = "Новое";
}
```

Рисунок 1 – Изменение состояния посредством get set

В программе предусмотрен механизм обработки событий, связанных с изменением состояния. Когда компонент переходит в состояние "Сломано", вызывается защищённый метод `OnComponentBroken`, который генерирует событие `ComponentBroken`. Это событие позволяет уведомить другие части программы о поломке, что может быть полезно для управления состоянием автомобиля или выполнения ремонтных процедур.

Абстрактная структура класса `Component` обеспечивает гибкость и переиспользование кода, позволяя другим компонентам наследовать базовый функционал. Благодаря использованию абстрактных классов и методов, дочерние классы могут реализовать свою специфическую логику, сохраняя при этом общую структуру и функциональность. Такой подход улучшает читаемость, масштабируемость и надёжность программы.

```
// Защитный метод для вызова события поломки компонента
// Срабатывает, если компонент сломался
Ссылка: 1
protected virtual void OnComponentBroken()
{
    ComponentBroken?.Invoke(this, EventArgs.Empty); // Генерация события ComponentBroken
}

// Событие поломки компонента, которое оповещает об изменении состояния на "Сломано"
public event EventHandler ComponentBroken;
```

Рисунок 2 – Защитный метод для вызова метода компонента

На рисунке 3 изображена диаграмма классов программы.

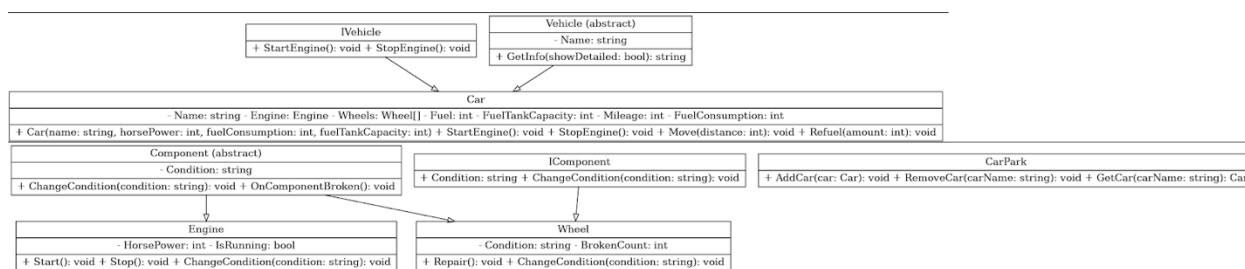


Рисунок 3 – Диаграмма классов

На рисунке 4 представлена структура оформления базового окна Windows Forms.

The screenshot shows a Windows Forms application window titled "Car Management". The window has a standard Windows title bar with minimize, maximize, and close buttons. The main area of the window is light gray and contains several controls:

- On the left side, there are four text labels with corresponding input fields:
 - "Введите имя авто:" followed by a text box.
 - "Введите мощность авто:" followed by a text box.
 - "Введите расход авто:" followed by a text box.
 - "Введите бак авто:" followed by a text box.
- Below these input fields are two buttons: "Создать авто" and "Удалить авто".
- Below the buttons is a large empty rectangular box.
- At the bottom left, there is a checkbox labeled "Подробно" and a button labeled "Показать".
- On the right side, there are five buttons stacked vertically: "Заправка", "Передвижение", "Запустить", "Выключить", and "Заменить".
- Next to the "Передвижение" button is a checkbox labeled "Высокая скорость".

Рисунок 4 – Оформление окна Windows Forms

4 Результаты работы программы с примерами

На рисунке 5 представлен результат работы программы показа состояния компонента.

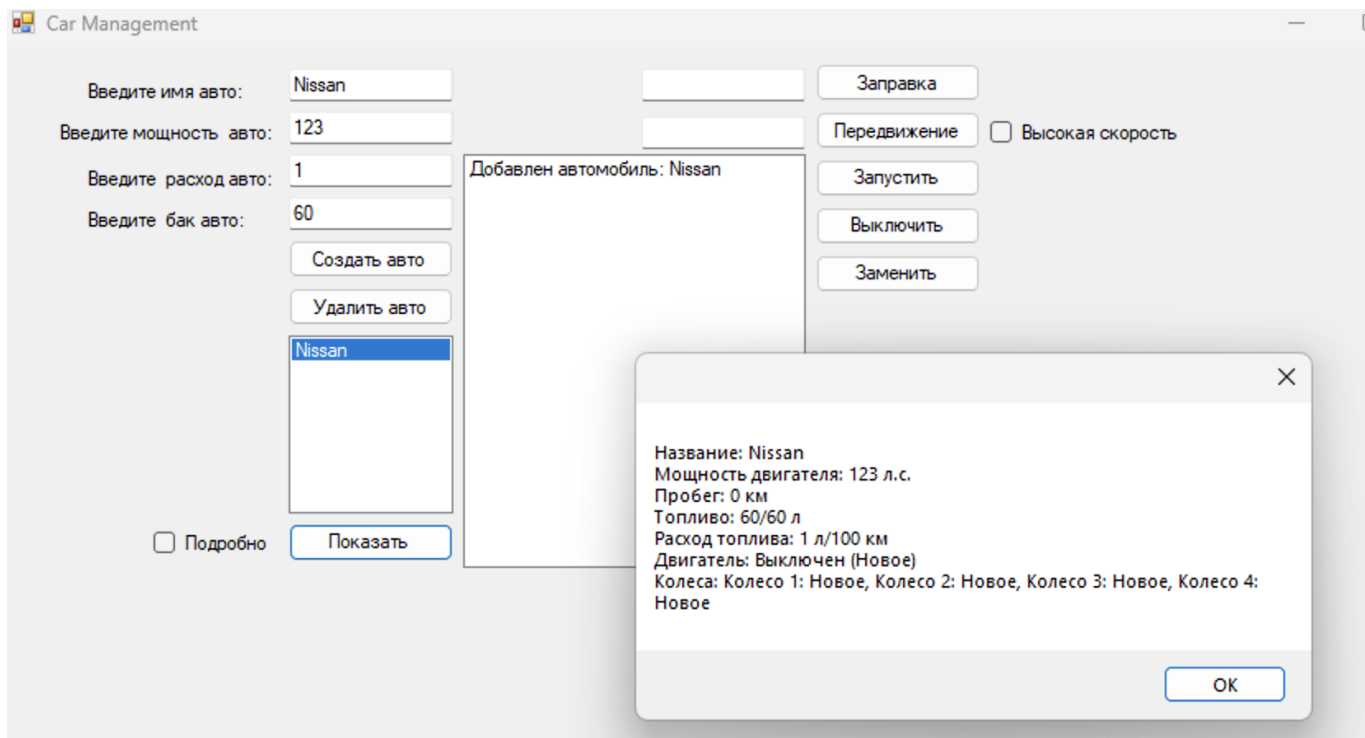


Рисунок 5 – Результат работы программы

5 Вывод

В ходе работы была изучена и реализована работа с абстрактными классами и методами. Абстрактный класс `Component` был использован для создания общей структуры компонентов автомобиля, включающей свойства и методы, общие для всех компонентов. Наследование позволило дочерним классам унаследовать эту базовую функциональность, сохраняя возможность для добавления или изменения специфических характеристик.

Также был рассмотрен механизм обработки событий. Событие `ComponentBroken`, реализованное в классе `Component`, демонстрирует, как можно эффективно оповещать другие части программы о поломке компонента. Это улучшает взаимодействие между частями системы и способствует модульности кода.

Использование абстрактного класса в сочетании с механизмом событий позволяет создавать гибкие и расширяемые системы, где базовая функциональность определена в одном месте, а дополнительные особенности реализуются в дочерних классах. Работа показала преимущества применения абстракции и событийного подхода в объектно-ориентированном программировании.

Файл «Component»

```
using System;

public abstract class Component
{
    // Абстрактный класс Component, который реализует общие методы и события для всех компонентов автомобиля
    // Свойство Condition хранит текущее состояние компонента, по умолчанию "Новое"
    public string Condition { get; protected set; } = "Новое";

    // Метод ChangeCondition для изменения состояния компонента
    // Проверяет, что состояние может быть только "Новое" или "Сломано"
    // При изменении состояния на "Сломано" вызывает событие поломки компонента
    public void ChangeCondition(string condition)
    {
        if (condition != "Новое" && condition != "Сломано")
            throw new ArgumentException("Неверное состояние компонента");

        Condition = condition;

        // Если состояние компонента "Сломано", генерируем событие поломки
        if (Condition == "Сломано")
        {
            OnComponentBroken();
        }
    }

    // Защитный метод для вызова события поломки компонента
    // Срабатывает, если компонент сломался
    protected virtual void OnComponentBroken()
    {
        ComponentBroken?.Invoke(this, EventArgs.Empty); // Генерация события ComponentBroken
    }

    // Событие поломки компонента, которое оповещает об изменении состояния на "Сломано"
    public event EventHandler ComponentBroken;
}
```